

WET2VO Web Technologies

Vorlesungsunterlagen

Wolfgang Hochleitner
wolfgang.hochleitner@fh-hagenberg.at

SS 2026

Vorlesung 8

Sessions und Authentifizierung

Bevor die Hauptthemen *Sessions* und *Authentifizierung* erläutert werden, ist es notwendig, zunächst über *HTTP-Response Header* und *Cookies* zu sprechen, da diese Dinge dort eine wesentliche Rolle spielen.

8.1 HTTP-Response Header

In Vorlesung 7 wurde ein Blick auf die Header geworfen, die bei der Anfrage (Request) an einen Webserver bei der GET- und POST-Methode übertragen werden. Die Antwort (Response), die der Webserver auf eine Anfrage eines Clients (Request) sendet, enthält ebenfalls solche *Header*. Wie auch in HTML enthalten Header verschiedene Metadaten, die dann vom Client ausgewertet werden können. In den Headern sind etwa ein Statuscode (200 – OK oder 404 – Not found), die Länge der übertragenen Daten, der Content-Type (z. B. `text/html`) oder auch Cookies (siehe Abschnitt 8.2) enthalten.

Diese Header-Daten werden in der Regel automatisch vom Webserver oder auch von PHP generiert und es gibt nur selten Grund zum Eingreifen. In manchen Fällen sollen aber explizit Anweisungen hinzugefügt werden. Dies kann über die Funktion `header($string)`¹ geschehen.

Die wichtigste Grundregel hierbei lautet: Header müssen *vor* jeglichem Output gesendet werden, d. h. vor eventuellem HTML-Markup oder auch `echo`-Aufrufen.

Folgende wichtige Dinge lassen sich mit der `header()`-Funktion erreichen:

8.1.1 Unterschiedliche Content-Types

Der normale Content-Type für PHP-generierte Ausgabe lautet `text/html`. Soll nun aber der Browser dazu bewegt werden, dass er die Ausgabe als reine Textdatei oder etwa auch als XML-Datei behandelt, so kann jeweils ein anderer Content-Type im Header mitgeschickt werden. Die entsprechenden Angaben lauten wie folgt:

```
1 header("Content-type: text/plain"); // Text
2 header("Content-type: text/xml"); // XML
```

¹<https://www.php.net/manual/de/function.header.php>

8.1.2 Weiterleitungen

Ein weiterer häufiger Einsatzgrund der Funktion `header()` ist das Weiterleiten auf einen andere URL. In diesem Fall müssen die Angabe `Location` sowie der neue URL im Header-String enthalten sein:

```
1 header("Location: https://www.fh-ooe.at/");  
2 exit();
```

Nach einer Weiterleitung sollte `exit()`;² aufgerufen werden. Dies terminiert das aktuelle PHP-Skript sofort. Da eine Weiterleitung auf einer andere Seite initiiert wurde, sollte auch die Verarbeitung der aktuellen am Server sofort gestoppt werden, um nicht unnötig Ressourcen zu verwenden. `exit()`; tut genau dies. Seit PHP 8.4 ist `exit()` eine echte Funktion (vorher war es ein Sprachkonstrukt). Sie sollte daher mit Klammern (also `exit()`) aufgerufen werden. Da die Funktion aber vom Parser nach wie vor speziell behandelt wird, ist es auch nach wie vor ohne Klammern (also `exit;`) möglich.

8.2 Cookies

Dass es sich bei HTTP um ein zustandsloses Protokoll handelt, wurde bereits im ersten Semester erläutert. Dies macht es besonders schwer, einen zusammenhängenden Zustand abzubilden. Denn um diesen zu erhalten, müssen Daten konsistent weitergegeben werden können.

Eine Möglichkeit, dies zu tun, sind versteckte Formularfelder. Alle wichtigen Daten werden von einem Seitenaufruf zum nächsten über solche versteckten Formularfelder weitergegeben. Da dieses Verfahren jedoch eine Sequenz von Formularen voraussetzt, ist es nicht sehr universell einzusetzen.

Eine zweite Möglichkeit ist das sogenannte URL-Rewriting (siehe auch Abschnitt 8.3), bei dem alle wichtigen Informationen als GET-Parameter von Seite zu Seite weitergegeben werden. Während diese Technik zwar überall funktioniert, kann die Weitergabe von vielen Parametern durchaus mühsam werden.

8.2.1 Allgemeines

Eine dritte Möglichkeit sind die sogenannten *Cookies*. Bei ihnen handelt es sich um eine sehr kleine, textbasierte Dateneinheit, die im Browser gespeichert wird. Sie werden mit einem Webserver assoziiert und auch bei jedem Request-Response-Zyklus mit versandt. Aus diesem Grund sind Cookies auch sehr klein und können nur beschränkte Datenmengen aufnehmen. Ein Cookie ist auf eine Größe von 4 Kilobyte beschränkt, weiters dürfen maximal 20 Cookies pro Server existieren.

Cookies dienen also quasi als Gedächtnisvariable, die einen Zustand zwischen einzelnen HTTP-Aufrufen speichern können. Sie dienen also primär der serverseitigen Verarbeitung und Informationsübertragung. Sollen nur clientseitige Informationen gespeichert werden, so empfiehlt sich der im ersten Semester besprochene Local Storage. Nichtsdestotrotz können Cookies aber auch clientseitig mittels JavaScript angesprochen werden.

²<https://www.php.net/manual/de/function.exit>

8.2.2 Cookie-Eigenschaften

Ein Cookie hat verschiedene Parameter:

- **name:** Der Name eines Cookies. Er kann beliebig gewählt werden und dient primär zur Identifikation.
- **value:** Der Wert eines Cookies, also die eigentlichen Daten, die lokal zwischengespeichert werden sollen.
- **expires:** Die Lebensdauer eines Cookies. Wird keine angegeben, so wird das Cookie beim Beenden des Browsers gelöscht. Ansonsten existiert es bis zum angegebenen Zeitstempel. Dieser wird in Sekunden seit dem 1.1.1970 (GMT) angegeben.
- **path:** Der Pfad am Webserver, für den das Cookie gelten soll. Standardmäßig ist dies der aktuelle Pfad inklusive aller Unterverzeichnisse.
- **domain:** Legt fest, für welche Domain das aktuelle Cookie vom Browser zurückgegeben werden soll. Standardmäßig wird immer der aktuelle Server angenommen.
- **secure:** Erlaubt es einzustellen, dass ein Cookie nur bei einer verschlüsselten HTTPS-Verbindung übertragen wird. Der Standardwert ist jedoch **false**, sodass Cookies auch bei unsicheren Verbindungen übertragen werden.
- **httponly:** Ist der Parameter auf **true** gestellt, ist das Cookie nur mehr über HTTP zugreifbar. Lokale Skriptsprachen wie JavaScript können nicht mehr darauf zugreifen. Diese Einstellung kann einen effektiven Schutz gegen XSS bieten.
- **samesite:** Legt fest, ob das Cookie bei Anfragen von anderen Webseiten (Cross-Site-Requests) mitgesendet wird.

8.2.3 Cookies setzen

Um ein Cookie mit den restlichen HTTP-Headern an den Browser zu senden, wird `setcookie($name [, $value, $expires, $path, $domain, $secure, $http])`³ verwendet. Die Parameter entsprechen den in Abschnitt 8.2.2 angeführten. Alle Parameter, außer der Name, sind optional. Sollen einzelne davon umgangen werden, kann einfach ein leerer String ("") dafür angegeben werden. Ausnahme ist `$expires`, das einen Integer verlangt, hier muss 0 gesetzt werden.

Seit PHP 7.3 gibt es eine alternative – und zu bevorzugende, weil flexiblere – Syntax zum Setzen von Cookies: `setcookie($name [, $value, [$options]])`. Der Parameter `$options` ist ein assoziatives Array mit möglichen Schlüsseln wie `expires`, `path`, `domain`, `secure`, `httponly` und `samesite`. Letzterer kann einen der folgenden Werte haben:

- **Strict:** Das Cookie wird *nur* bei sogenannten *Same-Site-Requests* mitgesendet, also wenn der Request direkt von derselben Domain stammt (z. B. bei Navigation oder Formularen innerhalb der eigenen Seite). Dies ist die restriktivste Einstellung.
- **Lax:** Wie **Strict**, aber Cookies werden auch bei *einfachen Cross-Site-GET-Requests* (z. B. durch einen Link von einer fremden Seite) mitgesendet. POST-Requests und andere komplexe Requests bleiben ausgeschlossen.

³<https://www.php.net/manual/de/function.setcookie>

- **None:** Das Cookie wird *auch bei Cross-Site-Requests* (z.B. über eingebettete Inhalte wie iFrames oder externe Skripte) mitgesendet. Voraussetzung ist, dass zusätzlich der **secure**-Parameter auf **true** gesetzt ist (nur über HTTPS).

Wird etwa ein YouTube-Video per **iframe** auf der eigenen Seite eingebunden, versucht YouTube ggf. ein Authentifizierungs- oder Personalisierungs-Cookie zu setzen. Damit dieses im Kontext einer Drittanbieter-Anfrage funktioniert, muss es mit den Attributen **samesite=None** und **secure=true** gesetzt sein. Viele Browser (z.B. Google Chrome ab Version 80) setzen **samesite=Lax** als Standardwert, wenn kein expliziter **samesite**-Wert angegeben ist.

Wichtig: Cookies werden an den Browser im Response-Header gesendet. Daher gilt für **setcookie()** dasselbe wie für **header()**: Der Aufruf muss *vor* jeglichem Output geschehen.

Session-Cookies

Sogenannte Session-Cookies sind der einfachste Fall eines Cookies. Sie haben kein Ablaufdatum und werden daher mit Schließen des Browsers wieder gelöscht. Ein Session-Cookie kann wie folgt erzeugt werden:

```
1 setcookie("user", "John Doe");
```

In diesem Fall wird unter dem Namen **user** der Wert **John Doe** abgespeichert.

Cookies mit Ablaufdatum

Um Cookies längerfristig zu speichern, muss ein Ablaufdatum gesetzt werden. Dieses muss in Sekunden seit dem 1.1.1970 vorliegen. Um einen sogenannten Unix-Zeitstempel zu erzeugen, kann die Funktion **time()**⁴ verwendet werden. Diese gibt den Zeitstempel für den aktuellen Augenblick zurück. Soll das Cookie von diesem Moment an genau eine Woche gültig sein, werden die entsprechenden Sekunden ($60 \cdot 60 \cdot 24 \cdot 7 = 604.800$) addiert (der Name des Cookies wird geändert, um sich vom Session-Cookie zu unterscheiden):

```
1 setcookie("username", "Jane Doe", ["expires" => time() + 604800]);
```

8.2.4 Cookies auslesen

Ruft der Browser eine Seite auf, für die ein gültiges Cookie in seinem lokalen Speicher existiert, so schickt er dieses automatisch an den Server. Dort kann es in PHP über ein weiteres superglobales Array, nämlich **\$_COOKIE**, ganz einfach ausgelesen werden. Der Schlüssel für den Zugriff ist der Name des Cookies. Um den Benutzer*innennamen aus dem zweiten der oben gesetzten Cookies auszulesen, ist folgender Code nötig:

```
1 echo "<p>Hello {$_COOKIE['username']}!</p>";
```

8.2.5 Cookies löschen

Um ein Cookie zu löschen, wird ein neues Cookie mit demselben Namen, aber leerem Wert erzeugt, dessen Ablaufdatum in der Vergangenheit liegt. Dafür wird von der

⁴<https://www.php.net/manual/de/function.time.php>

Funktion `time()` einfach ein Wert abgezogen (z. B. 43.200 Sekunden, was 12 Stunden entspricht). Dies veranlasst den Browser dazu, das Cookie lokal zu entfernen. Wichtig ist dabei, dass der vierte Parameter `$path` gleich wie beim Setzen des Cookies sein muss, da es sonst nicht funktioniert. Ebenso kann der Wert im `$_COOKIE` Superglobal mit `unset()` entfernt werden, um den bereits empfangenen Wert des Cookies für PHP in der aktuellen Datei nicht mehr verfügbar zu machen. Denn für den aktuellen Request kam das Cookie vom Browser noch mit und wird erst mit der Response der PHP-Datei gelöscht. Der nächste Request enthält es dann nicht mehr, weshalb der Eintrag in `$_COOKIE` erst dann leer wäre.

```
1 if (isset($_COOKIE["username"])) {  
2     setcookie("username", "", ["expires" => time() - 43200]);  
3     unset($_COOKIE["username"]);  
4 }
```

8.3 Sessions

Wie bereits am Beginn von Abschnitt 8.2 erwähnt, stellt das zustandslose HTTP Webentwickler*innen vor Herausforderungen, wenn es um die Aufrechterhaltung von Zuständen geht. Auch, wenn es, wie soeben gezeigt, Mechanismen wie Cookies gibt, um Werte zwischen zwei Seitenaufrufen zu speichern, so stellt das Speichern und Auslesen vieler Werte doch einen ständigen (mitunter lästigen) Mehraufwand dar.

8.3.1 Allgemeines

Die Lösung für dieses Problem ist das Session-Management. In PHP wird dazu eine eindeutige ID – die sogenannte *Session-ID* – vergeben. Diese muss bei jedem Request mitgeschickt werden, wodurch wiederum zusammengehörige Requests identifiziert werden können. Informationen (in Form von Variablen) sind über diese zusammengehörigen Requests hinweg persistent, bleiben also erhalten. Ähnlich wie Variablen innerhalb einer PHP-Datei verfügbar sind, sind diese Variablen dann auf allen Seiten, die durch die Session-ID als zusammengehörig identifiziert wurden, verfügbar. Derartige zusammengehörige Seiten werden im Allgemeinen als *Session* bezeichnet.

Die Session-ID

Um nun die Session-ID zwischen verschiedenen Seiten mitführen zu können, existieren drei verschiedene Mechanismen:

- Cookies: Die Session-ID wird in einem Cookie beim Browser gespeichert.
- URL-Codierung (URL-Rewriting): Die Session-ID wird in Form eines GET-Parameters an den URL gehängt.
- Versteckte Formularfelder: Die Session-ID wird in versteckten Formularfeldern mitgeschickt (selten bis gar nicht verwendet).

Cookies sind die Standardvariante, mit der die Session-ID übertragen wird. Das Cookie erhält dabei einen Namen – meist `PHPSESSID` – der Wert ist die generierte ID. Dies ist die einfachste Variante. Sie ist transparent für Benutzer*innen und einfach für Entwickler*innen, hat aber natürlich den Nachteil, dass Sessions bei ausgeschalteten

Cookies nicht verfügbar sind. Dies ist der Grund, warum viele Webseiten explizit darauf hinweisen, dass Cookies für einen reibungslosen Betrieb aktiviert sein müssen.

Damit der Webserver Cookies zur Übertragung der Session verwendet, muss die Einstellung `session.use_cookies = 1` in der Datei `php.ini` gesetzt sein. Dies ist jedoch in der Regel der Fall.

URL-Rewriting allgemein

URL-Rewriting ist eine Alternative zur Übertragung der Session-ID. Um es zu implementieren, muss jeder Link auf eine zur Session gehörende Folgeseite dynamisch um die Session-ID erweitert werden. Diese wird wie ein GET-Parameter mitgeführt: `http://localhost/sessions/index.php?PHPSESSID=104f342ad28f82f`

Diese Variante hat jedoch einige Nachteile. Wird die Seite als Bookmark gespeichert, wird die Session-ID mitgespeichert. Diese gilt jedoch zu einem späteren Zeitpunkt vielleicht nicht mehr. Ebenso muss bei einer Rückkehr auf die Seite (solange der Browser geöffnet bleibt) die Session-ID wieder an die URL gehängt werden, sonst geht die Session verloren. Des Weiteren stellt das Anhängen der Session-ID im Klartext ein Sicherheitsrisiko dar und begünstigt Angriffe wie etwa Session-Hijacking.

8.3.2 PHP-Sessions

PHP besitzt ein sehr klares und einfaches Session-Konzept. Eine Session wird am Beginn einer Seite gestartet oder fortgeführt. Danach können Variablen in der Session über ein superglobales Array gespeichert und ausgelesen werden. Die Übergabe der Session-ID geschieht standardmäßig über Cookies, ist aber auch per URL-Rewriting möglich. Am Server werden die Session-Daten unter der Session-ID gespeichert. Das Verzeichnis dazu wird in der `php.ini` durch die Variable `session.save_path` festgelegt.

Starten einer Session

Um eine Session zu starten wird die Funktion `session_start()`⁵ verwendet. Dieser Aufruf sollte ganz zu Beginn der Seite stehen, jedenfalls aber bevor Output erzeugt wurde.

Ist noch keine Session vorhanden, wird eine neue Session-ID generiert und gespeichert. Ist bereits eine Session-ID (in einem Cookie) vorhanden, wird diese Session fortgeführt.

Durch den Aufruf der Funktion werden gespeicherte Sessionvariablen verfügbar. Diese werden im superglobalen Array `$_SESSION` abgelegt. Dieses Array dient also als Sessionspeicher und kann aktiv beschrieben werden. Soll eine Variable in der Session gespeichert werden, muss nur ein neuer Eintrag im Array angelegt werden:

```
1 $_SESSION["user"] = "John Doe";
```

Beispiel: Reload-Counter

Im folgenden Beispiel wird ein simpler Reload-Counter implementiert. Er zählt, wie oft eine Seite neu geladen wurde. Dafür wird zunächst eine Session gestartet, dann wird die

⁵<https://www.php.net/manual/de/function.session-start.php>



Abbildung 8.1: Die typischen Schritte eines Kommunikationsablaufs mit Sessions: vom initialen Request über die Erstellung, Speicherung und Weitergabe der Session bzw. ihrer ID und der Wiederherstellung der damit assoziierten Daten.

Anzahl der Reloads als Variable in der Session (genauer gesagt im Array `$_SESSION`) gespeichert. Ist kein Wert vorhanden, wird er mit 0 neu initialisiert, sonst einfach um 1 erhöht.

```
1 <?php
2 session_start();
3
4 if (!isset($_SESSION["counter"])) {
5     $_SESSION["counter"] = 0;
6 } else {
7     $_SESSION["counter"]++;
8 }
9
10 echo "Anzahl Reloads: {"$_SESSION["counter"]}";
11 ?>
```

Ablauf einer Session

Am Beispiel des Reload-Counters lässt sich der Ablauf einer Session analysieren. Abbildung 8.1 stellt ihn dar.

Die folgenden 7 Schritte sind involviert.

1. Der initiale Request vom Browser wird an den Server gestellt. Die Session existiert noch nicht, darum ist auch kein Cookie involviert.
2. Am Server wird bei der aufgerufenen PHP-Seite (in der PHP-Engine) eine Session-ID erzeugt.
3. Der Server schreibt die Daten der Session in ein Session-File mit dieser ID. Die Session ist somit logisch am Server angelegt.
4. Die Response wird mit einem Cookie, das die Session-ID enthält, zurückgeschickt.
5. Das Cookie wird im Browser gespeichert.
6. Der nächste Request des Browsers schickt das Cookie mit der Session ID wieder zurück zum Server.
7. Dort wird die Session-ID mit der gespeicherten verglichen und die Daten können aus dem Session-File wiederhergestellt werden.

URL-Rewriting als Fallback, wenn Cookies nicht verfügbar sind

URL-Rewriting wurde früher als Fallback verwendet, wenn Cookies deaktiviert waren. Dabei wird die Session-ID über einen GET-Parameter im URL weitergegeben. Dies funktioniert über die Konstante `SID`. Sie enthält das komplette Schlüssel-Wert-Paar mit den Session-ID-Informationen. Aus Sicherheitsgründen (z. B. Gefahr von Session Hijacking durch mitgesendete URLs) wird dies heute nicht mehr empfohlen.

Moderne PHP-Versionen setzen in `php.ini` standardmäßig `session.use_only_cookies = 1`, wodurch URL-Rewriting gar nicht mehr automatisch funktioniert (`SID` enthält dann keinen Wert). Dieses Verhalten dient der Sicherheit und ist seit PHP 5.3 (2009) Standard.

Beenden einer Session

Um eine Session zu beenden, sind mehrere Schritte notwendig. Zwar löscht die Funktion `session_destroy()`⁶ alle, mit einer Session in Verbindung stehende Daten am Server, das globale Array `$_SESSION` sowie das Cookie zur Übertragung der Session-ID sind jedoch nicht betroffen. Diese müssen daher extra gelöscht werden.

Der komplette Vorgang zum Löschen einer Session sieht wie folgt aus. Dieser Code findet sich oft in einer eigenen Datei (z. B. `logout.php`).

```
1 session_start();
2
3 $_SESSION = [];
4
5 if (isset($_COOKIE[session_name()])) {
6     $params = session_get_cookie_params();
7     setcookie(
8         session_name(),
9         "",
10        [
11            "expires" => time() - 42000,
12            "path" => $params["path"],
13            "domain" => $params["domain"],
14            "secure" => $params["secure"],
15            "httponly" => $params["httponly"],
16            "samesite" => $params["samesite"],
17        ],
18    );
19 }
20
21 session_destroy();
```

Zunächst wird die Session mittels `session_start()` fortgeführt. Im darauffolgenden Schritt wird das superglobale Array `$_SESSION` gelöscht, indem es mit einem leeren Array überschrieben wird. Der dritte Schritt ist das Löschen des Session-Cookies. Dazu wird zunächst überprüft, ob es gesetzt ist. Wenn dies der Fall ist, werden mittels `session_get_cookie_params()`⁷ die einzelnen Informationen des Session Cookies (Ablaufdatum, Pfad, Domain, HTTP only und SameSite) abgefragt. Danach wird das Session Cookie mit einem ungültigen Cookie überschrieben. Dieses erhält den Namen des Session Cookies (abgefragt durch `session_name()`⁸), hat jedoch keinen Inhalt. Un-

⁶<https://www.php.net/manual/de/function.session-destroy.php>

⁷<https://www.php.net/manual/de/function.session-get-cookie-params.php>

⁸<https://www.php.net/manual/de/function.session-name.php>

gültig wird es dadurch, dass sein Ablaufdatum in der Vergangenheit liegt. Dazu wird vom aktuellen Zeitstempel einfach ein frei gewählter Sekundenbetrag subtrahiert. Der Browser erkennt dadurch, dass das Cookie abgelaufen ist und löscht es. Schließlich wird durch den Aufruf von `session_destroy()` die Session auch logisch am Server beendet.

8.3.3 Klassen und Sessions

Bei den Abschnitten über Vererbung und Interfaces in Vorlesung 4 wurde bereits gezeigt, dass PHP zwingend den Inhalt einer Basisklasse oder eines Interface kennen muss, bevor damit gearbeitet werden kann. Da Basisklasse und abgeleitete Klasse bzw. Interface meist in verschiedenen Dateien liegen, müssen diese mittels `include` oder `require` (ggf. mit Hilfe eines Autoloaders) zunächst eingebunden werden.

Auch bei der Arbeit mit Sessions kommt ein ähnliches Thema zum Tragen. Beim Session-Management werden alle Session-Variablen zur Übergabe zwischen den einzelnen Seiten in einer temporären Datei am Webserver abgespeichert. Dieser Vorgang wird Serialisierung genannt. Werden die Variablen auf der Folgeseite wiederhergestellt, wird von Deserialisierung gesprochen.

Nicht nur einfache Datentypen können serialisiert und deserialisiert werden. Auch Objekte können auf diese Art in der Session gespeichert werden. Die Wiederherstellung von Objekten ist jedoch komplexer als die eines einfachen Datentyps, da ein Objekt meist aus einer Reihe von verschiedenen Eigenschaften besteht. Nichtsdestotrotz kann PHP diesen Prozess vollständig automatisieren, sodass für Entwickler*innen kein Mehraufwand entsteht. Die einzige Voraussetzung ist, dass die Klassendefinition zum Zeitpunkt der Deserialisierung bekannt ist. Daher sollte diese am besten in eine eigene Datei ausgelagert werden und *vor* dem Aufruf von `session_start()` eingebunden werden.

Soll etwa ein Objekt der Klasse `Rectangle` in der Session gespeichert werden, so sollte die Klassendefinition in einer eigenen PHP-Datei namens `Rectangle.php` oder auch `Rectangle.inc.php` gespeichert sein:

```
1 // Rectangle.inc.php
2 class Rectangle implements GeometricComponentInterface
3 {
4     // ...
5 }
```

Erstreckt sich die Session etwa über zwei Dateien, so muss in beiden diese Datei vor dem Beginn/Weiterführen der Session eingebunden werden:

```
1 // SessionPage1.php
2 require("Rectangle.inc.php");
3 session_start();
4 // ...
```

```
1 // SessionPage2.php
2 require("Rectangle.inc.php");
3 session_start();
4 // ...
```

8.3.4 Session-Sicherheit und weitere Funktionen

Sessions stehen sehr oft in Zusammenhang mit geschützten oder sensiblen Daten, etwa im Falle eines Login-Mechanismus (siehe Abschnitt 8.4) oder auch bei Dingen wie Onlineeinkäufen. Daher sollten im Umgang mit Sessions einige Dinge beachtet werden. Darüber hinaus gibt es weitere Funktionen im Zusammenhang mit Sessions, die für die Arbeit hilfreich sein können.

Session-ID geheim halten

Die Session-ID sollte nach Möglichkeit immer geheim gehalten werden, da sie den Schlüssel zum Übernehmen einer Session („Session Hijacking“) darstellt. Darum sollte auf Dinge wie URL-Rewriting aus Sicherheitsgründen verzichtet werden. Auch das Verwenden einer sicheren Verbindung (über SSL) trägt zur sicheren Übertragung der Session-ID bei.

Session-ID wechseln

Die Session-ID sollte im Verlauf einer Applikation an wichtigen Stellen gewechselt, d. h. neu generiert werden. Dies verhindert Session Hijacking in Kombination mit der sogenannten „Session Fixation“, wo ein*e Angreifer*in eine Session-ID anfordert und diese einem Opfer mit einem Link etwa zum Login unterjubelt. Meldet sich der*die User*in an, so gilt die Session-ID als eingeloggt und der*die Angreifer*in hat Zugriff zum System. Wird die Session-ID jedoch nach einem Login automatisch neu generiert, bietet sich dieses Angriffsszenario nicht. PHP bietet mit `session_regenerate_id()`⁹ eine einfache Möglichkeit, die Session-ID zu wechseln.

Session benennen

Die Funktion `session_name()`¹⁰ erlaubt es, Sessions zu benennen bzw. den gesetzten Namen abzufragen. Der Standardwert lautet `PHPSESSID`, existieren jedoch auf einem Server zwei Webapplikationen (etwa ein Blog und ein Wiki), so kann es vorkommen, dass beide diesen Standardwert verwenden und Sessions daher (unfreiwillig) geteilt bzw. wiederverwendet werden. Um dies zu trennen, können beide Applikationen einen eindeutigen Sessionnamen wählen. Der Aufruf muss dabei *vor* `session_start()` erfolgen.

Status einer Session abfragen

Um zu erfragen, ob eine Session existiert oder die aktuellen Einstellungen überhaupt Sessions zulassen, kann `session_status()`¹¹ zum Einsatz kommen. Die Funktion liefert eine Ganzzahl zurück, die sich gegen die folgenden drei Konstanten vergleichen lässt:

1. `PHP_SESSION_DISABLED`: Sessions sind in der Datei `php.ini` deaktiviert.
2. `PHP_SESSION_NONE`: Sessions sind aktiviert, es existiert jedoch keine.
3. `PHP_SESSION_ACTIVE`: Sessions aktiviert und es gibt derzeit eine aktive.

⁹<https://www.php.net/manual/en/function.session-regenerate-id.php>

¹⁰<https://www.php.net/manual/en/function.session-name.php>

¹¹<https://www.php.net/manual/en/function.session-status.php>

Session-ID setzen oder abfragen

Mittels `session_id()`¹² kann die aktuelle Session-ID abgefragt oder auch selbst generiert werden. Für letzteres gibt es jedoch Auflagen hinsichtlich verwendeter Zeichen und Länge. Wird die Session-ID selbst generiert, muss dies vor dem Aufruf von `session_start()` geschehen.

Session-Einstellungen in `php.ini` überschreiben

Seit PHP 7.0 können beim Starten einer Session Session-Konfigurationswerte aus der Datei `php.ini` überschrieben werden. Dazu wird `session_start()` mit einem assoziativen Array aufgerufen, das die zu überschreibenden Parameter neu setzt. Im folgenden Beispiel wird eine Session gestartet in der die Lebensspanne eines Session-Cookies (bis zum Schließen des Browsers) statt des Standardwerts von 0 eine Stunde (3.600 Sekunden) beträgt.

```
1 session_start(["cookie_lifetime" => 3600]);
```

Die folgenden Werte können angepasst werden (jeweils ohne den Prefix `session.`): <https://www.php.net/manual/de/session.configuration.php>.

8.4 Authentifizierung mit PHP

Unter *Authentifizierung* (auch Authentifikation) im Web wird ganz allgemein der Nachweis einer behaupteten Eigenschaft einer Partei verstanden. Im Falle von Computersystemen wird dabei in der Regel die Identität einer Person oder eines Programms nachgewiesen. Den positiven Abschluss einer Authentifizierung nennt man auch Autorisierung und dieser gestattet wiederum im Fall von Computersystemen den Zugriff auf bestimmte Bereiche oder Ressourcen, die zuvor nicht erreichbar waren.

Ein gängiger Mechanismus zur Authentifizierung ist die Abfrage eines Benutzer*innennamens in Kombination mit einem Passwort. Andere Möglichkeiten sind die modernen Passkeys, Login-PINs, die Verwendung von Hardware-Token oder auch biometrische Merkmale wie Fingerabdrücke.

Im Folgenden wird der Schutz einer Seite mittels Authentifizierung durch eine typische Login-Seite mit Benutzer*innenname (in Form einer E-Mail-Adresse) und Passwort basierend auf Sessions und Redirects erläutert.

8.4.1 Funktionsweise und Ablauf

Eine Voraussetzung für die Implementierung mittels PHP ist der Browser-Redirect über HTTP-Header. Wie bereits in Abschnitt 8.1.2 besprochen, bewirkt dieser, dass der Browser vom PHP-Skript angewiesen werden kann, eine bestimmte Seite zu laden. Die Syntax zur Wiederholung:

```
1 header("Location: newSite.php");
```

Nach diesem Aufruf wird die Seite `newSite.php` angezeigt, als Statuscode wird *302 Redirect* angegeben. Vor dem Aufruf von `header()` darf keinerlei Ausgabe erfolgen. In

¹²<https://www.php.net/manual/en/function.session-id.php>

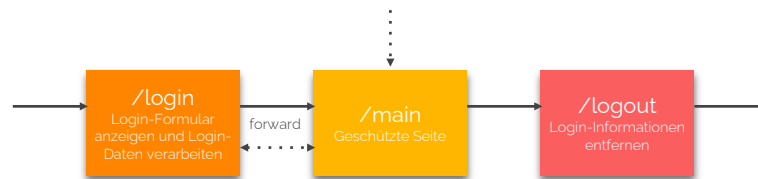


Abbildung 8.2: Die logische Abfolge der Routenaufrufe ist mit den durchgehenden Pfeilen dargestellt. Mögliche Abweichungen mit den gestrichelten. Wird `GET /main` direkt ohne Login aufgerufen, wird sofort auf `GET /login` weitergeleitet. Wird der Login dort korrekt durchgeführt, geschieht die Weiterleitung auf `GET /main` und die Seite darf betreten werden.

einem allgemeineren Fall kann auch ein vollständiger URL angegeben werden. In diesem Beispiel wird auf eine Seite am gleichen Webserver weitergeleitet, die auch im selben Verzeichnis liegt, wie die Datei, von der aus der Redirect erfolgt.

Die vorgestellte *PHP-Authentifizierung* ist eine mit `fhooe/router` realisierte Applikation. Die folgenden vier Routen sind involviert:

- `GET /login` & `POST /login`: Zeigt ein Formular mit den Feldern für Benutzer*innenname in Form einer E-Mail-Adresse und einem Passwort für einen Login an. Beim Abschicken wird in `POST /login` überprüft, ob die Angaben korrekt sind und ggf. auf die Route der Hauptseite `GET /main` weitergeleitet. Bei falschen Angaben wird auf der Seite verweilt.
- `GET /main`: Die Hauptseite mit dem eigentlichen Inhalt. Sie darf nur betreten werden, wenn ein korrekter Login durchgeführt wurde. Daher wird zunächst immer überprüft, ob dieser stattgefunden hat. Wenn nicht, wird sofort auf `GET /login` umgeleitet.
- `GET /logout`: Führt einen vollständigen Logout durch und entfernt alle Informationen über einen korrekten Login.

Der Ablauf zwischen diesen drei Routen ist in Abbildung 8.2 dargestellt.

8.4.2 Erzeugen der Login-Daten

Bevor mit dem Login begonnen werden kann, müssen überhaupt *Login-Daten* vorhanden sein, auf die später überprüft werden kann. In der Regel können sich Benutzer*innen auf einer Registrierungsseite einen Benutzer*innennamen (in Form einer E-Mail-Adresse) mit dazugehörigem Passwort angeben. Beide Werte müssen nun im System gespeichert werden, damit später beim Login überprüft werden kann, ob die korrekte Kombination eingegeben wurde. Der Benutzer*innennamen kann dabei im Klartext gespeichert werden, das Passwort sollte jedoch *niemals* in lesbarer Form gespeichert werden. Egal ob die Speicherung dabei in einer Datenbank oder einer Textdatei erfolgt. Klartextpasswörter stellen ein Sicherheitsrisiko dar, falls sich Angreifer*innen unerlaubten Zugriff auf den Speicherort verschaffen. Daher ist eine Verschlüsselung unumgänglich.

Passwörter verschlüsseln

Die derzeit sicherste Methode zur *Verschlüsselung von Passwörtern* ist die Verwendung der Funktion `password_hash($password, $algo, $options)`¹³. Der erste Parameter ist das zu hashende Passwort, der zweite spezifiziert den zu verwendenden Algorithmus. Dabei empfiehlt es sich, entweder `PASSWORD_DEFAULT` anzugeben (was den Standard-Algorithmus der aktuellen PHP-Version, meist `bcrypt`, verwendet) oder direkt auf moderne Standards wie `PASSWORD_ARGON2ID` zu setzen. Das Ergebnis ist ein Hashwert, der den Salt und die Informationen über den verwendeten Algorithmus bereits enthält, sodass die Gegenprüfung ganz ohne zusätzliche Datenbankfelder erfolgen kann.

Um einen mit dieser Funktion generierten Passwort-Hash zu überprüfen, existiert die Funktion `password_verify($password, $hash)`.¹⁴ Der erste Parameter ist dabei das zu überprüfende Passwort im Klartext, das zweite Argument ist der gehashte Wert aus der Datenbank. Die Funktion führt den Hash-Vorgang erneut durch; wenn das Ergebnis übereinstimmt, gibt sie `true`, ansonsten `false` zurück.

Um zu überprüfen, ob der im gespeicherten Hash verwendete Algorithmus oder dessen Parameter noch aktuell sind, kann `password_needs_rehash($hash, $algo)`¹⁵ nach dem erfolgreichen Login verwendet werden. Falls sich die Anforderungen geändert haben (z. B. Server-Upgrade oder höhere Sicherheitsrichtlinien), gibt die Funktion `true` zurück. Das Passwort kann dann aus dem Klartext der aktuellen Login-Anfrage direkt neu verschlüsselt und in der Datenbank aktualisiert werden. Somit ist stets die höchste Sicherheit gewährleistet.

Sollen Informationen über einen mit `password_hash()` erzeugten Hash abgefragt werden, kann `password_get_info($hash)`¹⁶ zum Einsatz kommen. Es gibt ein assoziatives Array mit den Schlüsseln `algo`, `algoName` und `options` zurück, welches die verwendeten Speicher- und Zeitkosten des Hash-Vorgangs offenlegt.

Diese nativen PHP-Passwort-Funktionen sollten immer jeglicher selbstgeschneider-ten Lösung vorgezogen werden, um sicheres Passwortmanagement zu gewährleisten.

8.4.3 Der Login-Vorgang

Beim *Login-Vorgang* wird ein Benutzer*innenname in Form einer E-Mail-Adresse mit Passwort abgefragt. Stimmen beide mit dem gespeicherten Datensatz überein, darf die Hauptseite betreten werden. Wird die Hauptseite direkt betreten, ohne vorher einen Login durchgeführt zu haben, wird auf die Login-Seite verwiesen. In den folgenden Abschnitten werden die einzelnen Codestücke beschrieben, die zum Ablauf notwendig sind.

Definitionen der Routen

Zunächst werden in der Hauptdatei `index.php` (im Verzeichnis `public`) die Routen angelegt. Neben der Hauptroute `GET /`, welche der Startpunkt des Ablaufs ist, werden vier weitere Routen angelegt:

¹³ <http://php.net/manual/de/function.password-hash.php>

¹⁴ <https://www.php.net/manual/de/function.password-verify.php>

¹⁵ <https://www.php.net/manual/de/function.password-needs-rehash.php>

¹⁶ <https://www.php.net/manual/de/function.password-get-info.php>

- GET /login: Zeigt das Login-Formular an.
- POST /login: Verarbeitet das Login-Formular und überprüft, ob korrekte Login-Daten vorhanden sind.
- GET /main: Zeigt die Hauptseite an.
- GET /logout: Entfernt alle gültigen Login-Informationen.

Die Definitionen hierfür lauten wie folgt:

```

1 $router->get("/login", function () use ($latte, $router) {
2     // Login-Formular anzeigen
3 });
4
5 $router->post("/login", function () use ($latteig, $router) {
6     // Login-Formular verarbeiten
7 });
8
9 $router->get("/main", function () use ($latte, $router) {
10    // Geschützte Hauptseite anzeigen
11 });
12
13 $router->get("/logout", function () use ($router) {
14    // Ausloggen
15 });

```

Die ersten drei Routen sollen eine Response (HTML-Output) anzeigen und bekommen daher das Latte-Objekt übergeben, damit sie die entsprechenden Templates ausgeben können. Alle Routen bekommen das Router-Objekt mitgegeben, damit sie Weiterleitungen auf andere Routen durchführen können.

Überprüfung der Login-Daten

Die Route GET /login startet den eigentlichen Login-Vorgang. Sie muss zunächst lediglich ein Latte-Template mit dem Login-Formular ausgeben:

```

1 $router->get("/login", function () use ($latte, $router) {
2     $latte->render("login.latte");
3 });

```

Das Formular spezifiziert in seinem action-Attribut, dass die Route POST /login aufgerufen wird. Somit findet die gesamte Verarbeitung in der Handler-Funktion dieser Route statt:

```

1 $router->post("/login", function () use ($latte, $router) {
2     $login = new Login($latte, $router);
3     $login->isValid();
4     $login->displayOutput();
5 });

```

Zunächst wird ein Objekt der Klasse Login angelegt. Diese Klasse kapselt alle Funktionalitäten, um die Eingaben aus dem Formularfeld zu validieren, ggf. Fehlermeldungen anzuzeigen und bei Erfolg auf die geschützte Seite weiterzuleiten.

Die Klasse hat eine Eigenschaft `$pdo` für die PDO-Datenbankverbindung, ein Array `$messages`, das mögliche Fehlermeldungen speichert, ein Latte-Objekt `$latte`, welches die Templatefunktionalitäten beinhaltet und ein Router-Objekt `$router` für Weiterleitungen.

Im Konstruktor der Klasse werden die übergebenen Latte- und Router-Objekte gespeichert und eine Datenbankverbindung aufgebaut.

```

1 final class Login
2 {
3     private PDO $pdo;
4     private array $messages = [];
5     private Engine $latte;
6     private Router $router;
7
8     public function __construct(Engine $latte, Router $router)
9     {
10         $this->latte = $latte;
11         $this->router = $router;
12         $this->initDB();
13     }
14
15     // ...
16 }

```

In der Methode `isValid()` muss nun überprüft werden, ob die eingegebenen Daten korrekt sind:

```

1 public function isValid(): void
2 {
3     if (Utilities::isEmptyString($_POST["email"])) {
4         $this->messages["email"] = "Please enter your email address.";
5     } elseif (!Utilities::isEmail($_POST["email"])) {
6         $this->messages["email"] = "Please enter a valid email address";
7     }
8     if (Utilities::isEmptyString($_POST["password"])) {
9         $this->messages["password"] = "Please enter your password.";
10    }
11
12    if (count($this->messages) === 0) {
13        if (!$this->authenticateUser()) {
14            $this->messages["login"] = "Invalid account email address or password.";
15        } else {
16            $this->business();
17        }
18    }
19 }

```

Beide Felder – die E-Mail-Adresse und das Passwort – dürfen zunächst nicht leer sein. Bei der E-Mail-Adresse wird weiters überprüft, ob es sich auch um eine korrekte Format handelt. Dies wird mit einer statischen Hilfsfunktion aus der Klasse `Utilities` erledigt.

Sind diese Punkte sichergestellt, wird die Methode `authenticateUser()` aufgerufen, die überprüft, ob die angegebene E-Mail-Passwort-Kombination in den gespeicherten Login-Daten enthalten ist. Gibt die Methode `true` zurück, sind beide Angaben korrekt, bei `false` sind die Daten falsch. Aus Sicherheitsgründen wird nicht angegeben, ob E-Mail-Adresse oder Passwort oder beides falsch bzw. nicht vorhanden ist, sondern nur eine generelle Fehlermeldung.

Die Methode `authenticateUser()` fragt nun aus der Tabelle `user` den Datensatz ab, dessen E-Mail-Adresse mit der eingegebenen übereinstimmt. Erhält das Ergebnis einen

Datensatz, so wird das Passwort dieses Datensatzes mit dem eingegebenen verglichen. Dabei kommt, wie in Abschnitt 8.4.2 beschrieben, `password_verify()` zum Einsatz. Gibt die Funktion `true` zurück, handelt es sich um das korrekte Passwort und der*die Benutzer*in gilt als korrekt authentifiziert.

Mit einer zusätzlichen Abfrage wird noch überprüft, ob das Passwort neu verschlüsselt werden muss. Mit Hilfe von `password_needs_rehash()` wird getestet, ob der gespeicherte Hash noch den Anforderungen an den aktuellen gewünschten Algorithmus (hier `PASSWORD_DEFAULT`) erfüllt. Ist dies der Fall, passiert nichts, ansonsten wird das Passwort mit neu gehashed und der Eintrag in der Datenbank mittels `updateUser()` aktualisiert.

Ein*e User*in gilt dann *nicht* als korrekt authentifiziert, wenn die E-Mail-Adresse nicht in den gespeicherten Datensätzen vorhanden war oder das eingegebene Passwort nicht mit dem gespeicherten übereinstimmt.

```

1 private function authenticateUser(): bool
2 {
3     $query = "SELECT user_id, email, password FROM user WHERE email = :email";
4     $params = [":email" => $_POST["email"]];
5
6     $statement = $this->pdo->prepare($query);
7     $statement->execute($params);
8     $row = $statement->fetch();
9
10    if ($row && password_verify($_POST["password"], $row["password"])) {
11        if (password_needs_rehash($row["password"], PASSWORD_DEFAULT)) {
12            $password = password_hash($_POST["password"], PASSWORD_DEFAULT);
13            $this->updateUser($row["user_id"], $password);
14        }
15        return true;
16    }
17    return false;
18 }

```

Weiterleitung bei erfolgreichem Login

Wurden E-Mail-Adresse und Passwort als gültig anerkannt, erfolgt in `business()` die Weiterleitung auf den geschützten Bereich, z. B. Route `GET /main`. Dafür wird auf die Methode `redirectTo()` der Klasse `Router` zurückgegriffen. Diese verwendet die PHP-Funktion `header()` gefolgt von einem `exit()`-Aufruf und erlaubt als Parameter eine einfache Routen-Angabe. Diese wird in einen vollen URL aufgelöst und die Weiterleitung erfolgt. Als Wert wird hier `/main` übergeben.

Vor der eigentlichen Weiterleitung werden noch wichtige Werte in die Session geschrieben. Zunächst die E-Mail-Adresse des*der angemeldeten Benutzers*Benutzerin. Somit ist diese auf den Folgeseiten verfügbar. Weiters wird durch Aufruf der statischen Methode `generateLoginToken()` aus der `Utilities`-Klasse noch ein kryptografisch sicheres Token generiert. Die Methode verwendet die Funktion `random_bytes(32)`, um 32 zufällige, kryptografisch sichere, Bytes zu erzeugen. Die erzeugten Bytes werden anschließend mit der Funktion `bin2hex()` in eine hexadezimale Darstellung umgewandelt, um sicherzustellen, dass das Token nur aus lesbaren Zeichen besteht. Später wird auf das Vorhandensein dieses Tokens in der Session überprüft, um als eingeloggt zu gelten.

Um die Sicherheit noch weiter zu erhöhen könnte das Token zusätzlich noch clientseitig (etwa in einem versteckten Formularfeld oder in einem Header) übertragen werden. Solche *Client-synchronisierten* Token müssen dann auch validiert (verglichen) werden, bieten aber erweiterten Schutz gegen Session-Fixation, Session-Hijacking oder CSRF. Solange die Session sicher ist, ist dieser erweiterte Mechanismus nicht zwingend nötig.

Die zuvor erwähnte Methode `business()` in der Klasse `Login` sieht wie folgt aus:

```
1 protected function business(): void
2 {
3     $_SESSION["email"] = $_POST["email"];
4
5     $_SESSION["loginToken"] = Utilities::generateLoginToken();
6
7     $this->router->redirectTo("/main");
8 }
```

Die verwendete Hilfsmethode ist in `Utilities.php` wie folgt definiert:

```
1 public static function generateLoginToken(): string
2 {
3     return bin2hex(random_bytes(32));
4 }
```

Da ein Login von einem einzigen Wert in der Session abhängt, ist es wichtig, diese gut zu sichern. So sollte das Session-Cookie nur über HTTPS übertragen und zugreifbar sein und die Same-Site-Policy auf Strict gesetzt sein. Dies kann beim `session_start()`-Aufruf als Optionen in einem Array spezifiziert werden:

```
1 session_start([
2     "cookie_secure" => true,
3     "cookie_httponly" => true,
4     "cookie_samesite" => "Strict",
5 ]);
```

Browser sehen `localhost` als sichere Quelle an, weshalb dies auch über `http://localhost:8080` funktioniert. Auf nicht-lokalen Webservern muss HTTPS verwendet werden, sonst wird das Cookie nicht mit dem Request an den Server zurückgeschickt und der gesamte Ablauf schlägt fehl.

Zusätzliche Sicherheit bietet das Beschränken des Cookies auf Domain und Pfad, das (Um-)Benennen des Session-Cookies (um zu verhindern in mehreren Applikationen am Server eingeloggt zu sein, wenn der Login-Mechanismus derselbe ist) und das Erneuern der Session-ID nach erfolgreichem Login mittels `session_regenerate_id()`.

Anzeige von Fehlermeldungen bei erfolglosem Login

War der Login erfolgreich, erfolgte aufgrund der Weiterleitung auf die geschützte Seite keinerlei Ausgabe. Gibt es jedoch Probleme, weil die Benutzer*innendaten nicht mit den gespeicherten übereinstimmen, so muss wieder das Login-Formular angezeigt werden. Dies geschieht in der Methode `displayOutput()`, die nach `isValid()` aufgerufen wird und eben nur dann auch tatsächlich ausgeführt wird, wenn die Überprüfung nicht erfolgreich war.

Die Methode zeigt das Latte-Template des Login-Formulars an, übergibt aber zusätzlich die bereits eingegebene E-Mail-Adresse, welche in `value`-Attribut des Eingabefeldes eingefügt wird. Somit ist die Adresse bereits vorausgefüllt. Weiters wird das

Array mit den Fehlermeldungen übergeben, welches diese im Template anzeigt, wenn `messages` übergeben wurde.

```
1 public function displayOutput(): void
2 {
3     $this->latte->render("login.latte", [
4         "email"     => $_POST["email"],
5         "messages" => $this->messages,
6     ]);
7 }
```

Absichern logingeschützter Seiten

Der Login-Mechanismus ist soweit komplett, jedoch könnte immer noch die Route der zu schützenden Seite (`GET /main`) ohne Login erreicht werden, da dort noch nicht überprüft wird, ob ein Login stattgefunden hat. Dies wird dadurch erreicht, dass vor dem Anzeigen des Templates der Hauptseite eine Abfrage auf das in der Session gespeicherten Login-Token getätigt wird. Ist dieses nicht vorhanden, wird zurück auf die Login-Seite geleitet, noch bevor die Hauptseite angezeigt wird. Diese Funktionalität ist als statische Methode `requireLoggedIn()` in einer Klasse `RouteGuard` angelegt.

Der Aufruf sieht wie folgt aus:

```
1 $router->get("/main", function () use ($latte, $router) {
2     RouteGuard::requireLoggedIn("loginToken", $router, "/login");
3     $latte->render("main.latte");
4 });
```

Die Methode ist folgendermaßen implementiert:

```
1 public static function requireLoggedIn(string $tokenName, Router $router, string
    $exitRoute): void
2 {
3     if (!isset($_SESSION[$tokenName])) {
4         $router->redirectTo($exitRoute);
5     }
6 }
```

Eine ähnliche Absicherung kann bei Bedarf auch für die Route `GET /login` verwendet werden. Sind User*innen eingeloggt, macht der Aufruf der Login-Seite wenig Sinn. Daher kann mit der umgekehrten Bedingung bei einem erfolgreichen Login direkt wieder auf die Hauptseite weitergeleitet werden. Dies ist in der Methode `requireNotLoggedIn()` in der Klasse `RouteGuard` implementiert:

```
1 public static function requireNotLoggedIn(string $tokenName, Router $router, string
    $exitRoute): void
2 {
3     if (isset($_SESSION[$tokenName])) {
4         $router->redirectTo($exitRoute);
5     }
6 }
```

Damit ändert sich die Routendefinition für `GET /login` wie folgt:

```
1 $router->get("/login", function () use ($latte, $router) {
2     RouteGuard::requireNotLoggedIn("loginToken", $router, "/main");
3     $latte->render("login.latte");
4 });
```

Logout

Benutzer*innen gelten so lange als eingeloggt, so lange die Session bzw. der Hashwert darin existiert. Während dieser Zeit können sie sich nicht erneut anmelden, da ja sofort auf die Hauptseite weitergeleitet würde, wenn ein korrekter Login existiert. Um den*die Benutzer*in abzumelden, muss die Session gelöscht werden, dies bewirkt automatisch einen *Logout*.

Die Route GET `/logout` enthält den notwendigen Code. Sie muss keinerlei HTML-Output erzeugen, sondern nur die Session zerstören und danach auf eine andere Route weiterleiten. Das Löschen der Session geschieht wie in Abschnitt 8.3.2 beschrieben. Zunächst wird das superglobale Session-Array gelöscht, dann das Session-Cookie überschrieben und schließlich die Session logisch entfernt.

```
1 $router->get("/logout", function () use ($router) {  
2     $_SESSION = [];  
3     if (isset($_COOKIE[session_name()])) {  
4         setcookie(session_name(), "", [  
5             "expires" => time() - 86400,  
6             "path"     => "/",  
7         ]);  
8     }  
9     session_destroy();  
10    $router->redirectTo("/");  
11 });
```

8.5 Weiterführende Literatur

Die Themen HTTP-Request-Header, Cookies und Sessions werden sehr ausführlich in [1] behandelt. Jedoch deckt das Buch dabei nur unwesentlich mehr Details ab, als in dieser Vorlesung vorgestellt wurden.

[2, 3] widmen den Themen Sessions und Cookies jeweils ganze Kapitel, die sowohl eine Einführung bieten als auch konkrete Beispiele besprechen.

Literatur zum Thema Authentifizierung im Web ist sehr dünn gestreut.

[2, 3] diskutieren beide jeweils ein PHP-Login-System, welche mit Datenbanken arbeiten. Sie funktionieren auf einer ähnlichen Basis, ist jedoch etwas rudimentärer implementiert.

Quellenverzeichnis

- [1] Kevin Tatroe und Peter MacIntyre. *Programming PHP. Creating Dynamic Web Pages*. 4. Aufl. Sebastopol, USA: O'Reilly, 31. März 2020 (siehe S. 8-19).
- [2] Thomas Theis. *Einstieg in PHP 8 und MySQL*. 15. Aufl. Bonn, Deutschland: Rheinwerk Computing, 5. Apr. 2023 (siehe S. 8-19).
- [3] Christian Wenz und Tobias Hauser. *PHP 8 und MySQL. Das umfassende Handbuch*. 5. Aufl. Bonn, Deutschland: Rheinwerk Computing, 3. Mai 2024 (siehe S. 8-19).